

Mexi Banco con Docker compose

1. Clonación del repositorio

Se utilizó Git para descargar el proyecto: `git clone --branch v05.1: https://github.com/OscarRuiz21/mexi-banco.git`

Preparación del entorno

Docker Desktop debe estar iniciado para que Docker Compose pueda crear y ejecutar los contenedores y Postman actúa como cliente HTTP para enviar solicitudes al servicio publicado en el puerto 8080. `mexi-banco-v05.1.postman_collection.json` proporciona las peticiones y las pruebas automáticas.

2. Arranque con Docker Compose

```
docker compose up --build -d
```

```
docker compose ps
```

- `up` crea e inicia los servicios definidos en Compose
- `--build` construye la imagen de la aplicación a partir del Dockerfile
- `-d` deja los contenedores ejecutándose en segundo plano.
- `ps` muestra su estado.

El termino de la compilación, la terminal mostró `app` y `db` como `healthy`: `Healthy` nos indica que superó la comprobación configurada y `Health: starting` es un estado transitorio mientras se realizan las primeras comprobaciones.

3. Pruebas de la API con Postman

- **Salud y apertura de cuentas**

La consulta `GET /actuador/health` respondió 200 OK con status UP. La apertura mediante `POST /cuentas` respondió 201 Created: Ana recibió un saldo inicial de 1000. La consulta `GET /cuentas/{clabe}` recuperó la cuenta de Ana con saldo 1000.

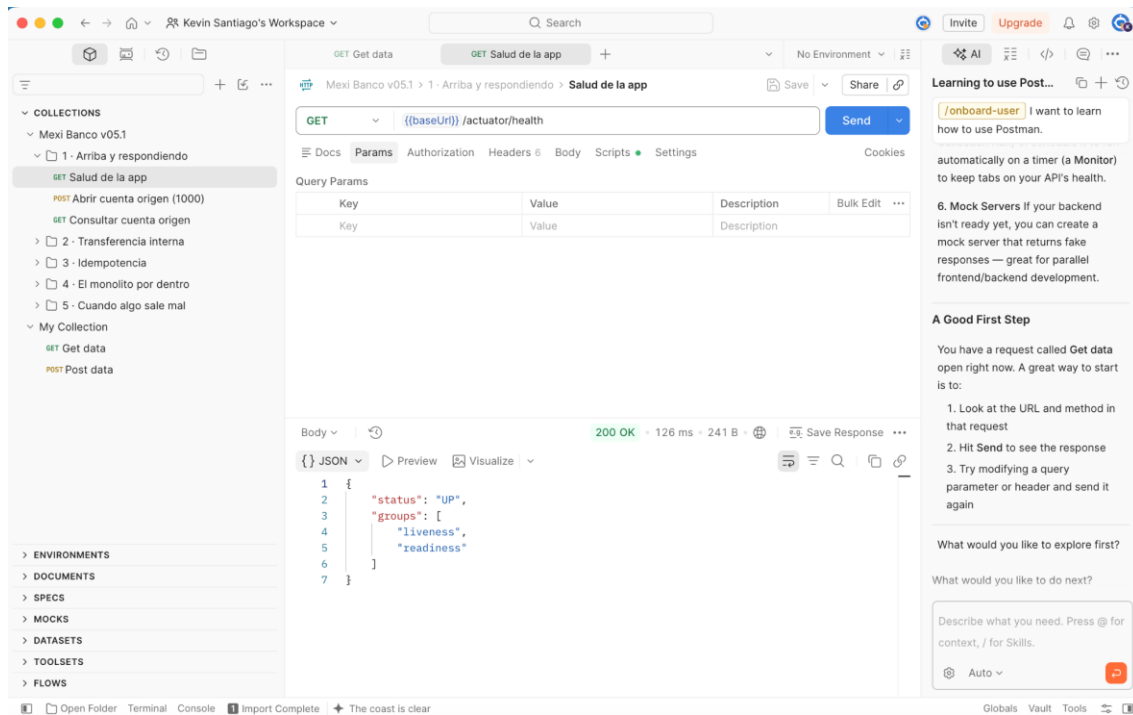


Figura 1. Salud de la aplicación. GET /actuator/health devuelve 200 OK y status UP.

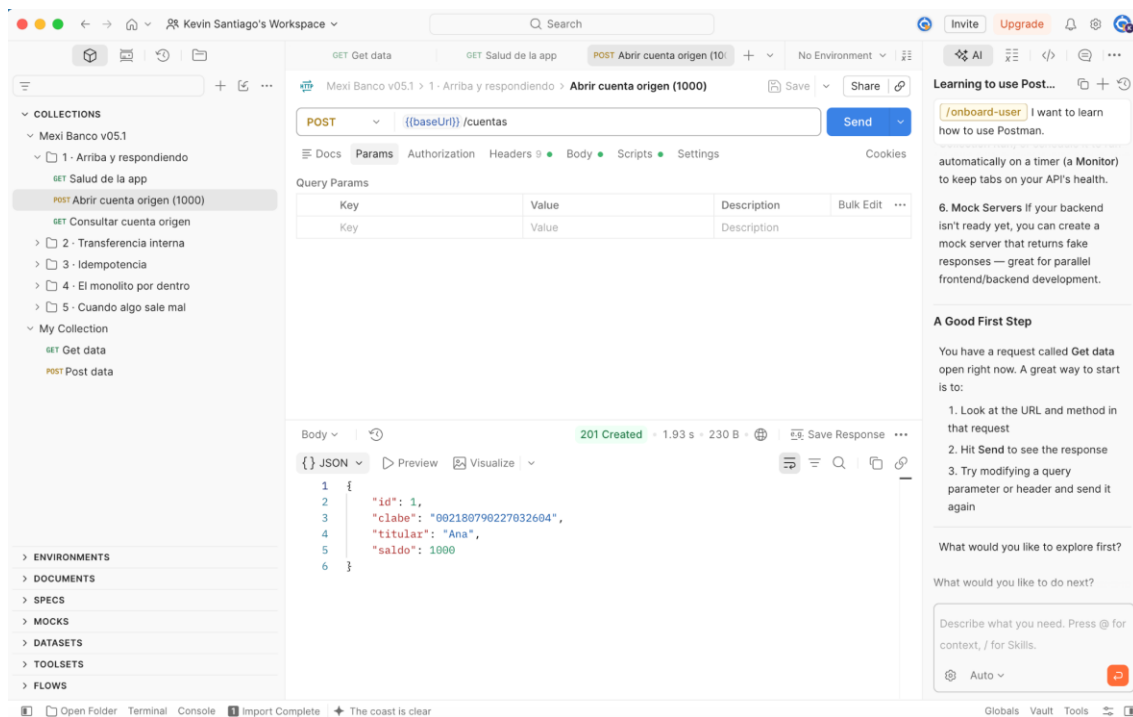


Figura 2. Apertura de la cuenta origen. POST /cuentas devuelve 201 Created.

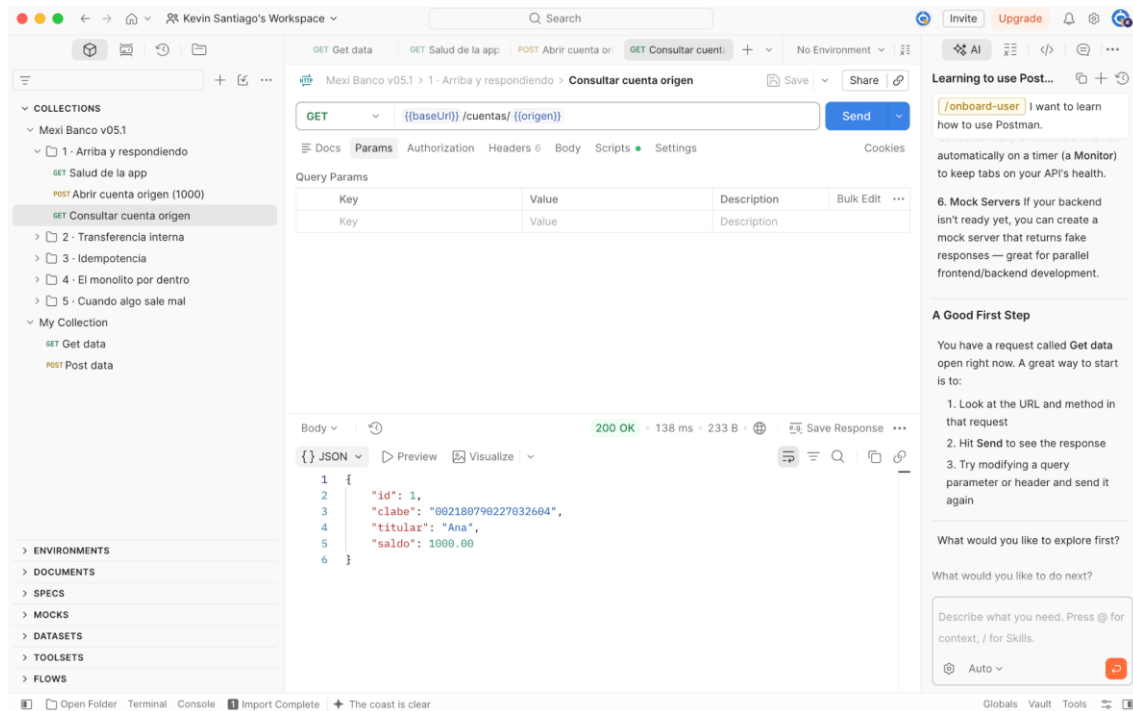


Figura 3. Consulta de la cuenta origen. La consulta responde 200 OK y recupera el saldo inicial de 1000.

• Transferencia interna

Se envió una transferencia de 200 desde la cuenta origen hacia la cuenta destino (Beto con un monto inicial de 500). La API respondió 201 Created y devolvió un registro con su identificador. Después se consultaron ambas cuentas y la transferencia.

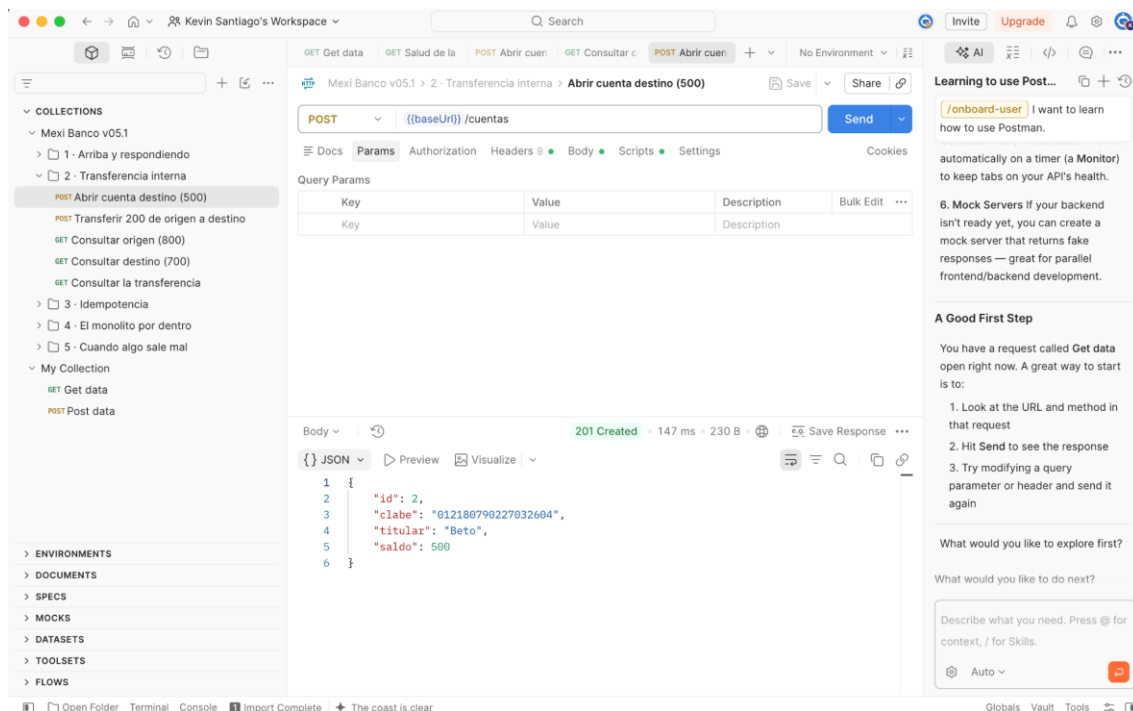


Figura 4. Apertura de la cuenta destino.

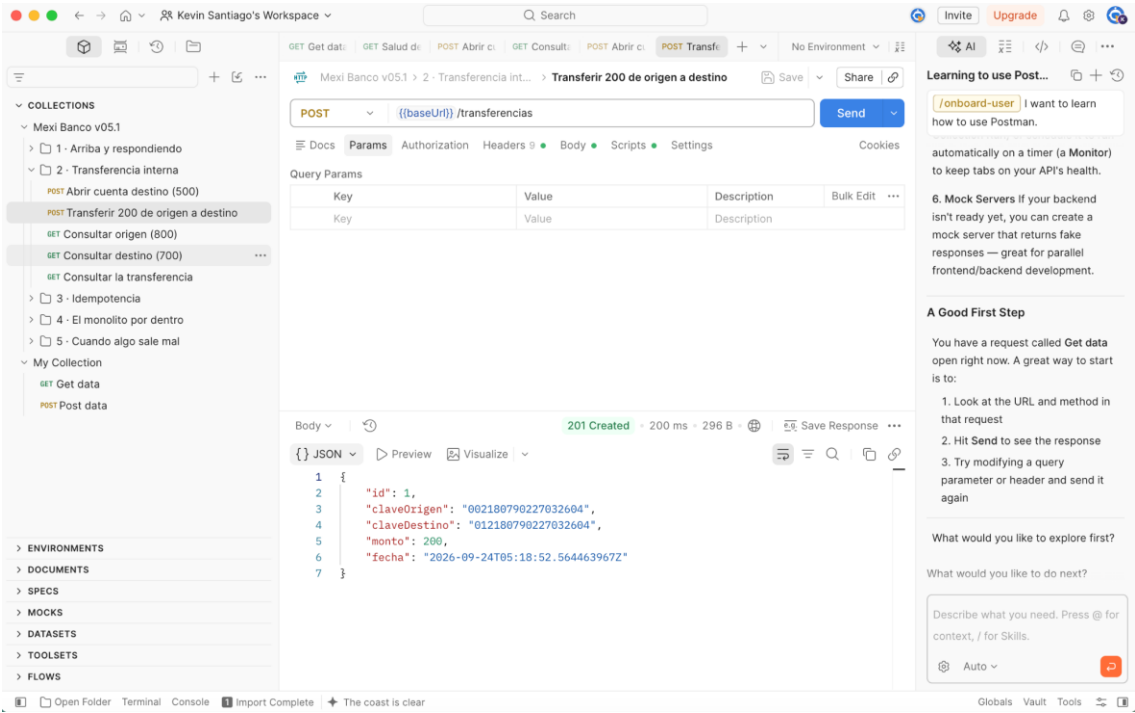


Figura 5. Transferencia interna de 200.

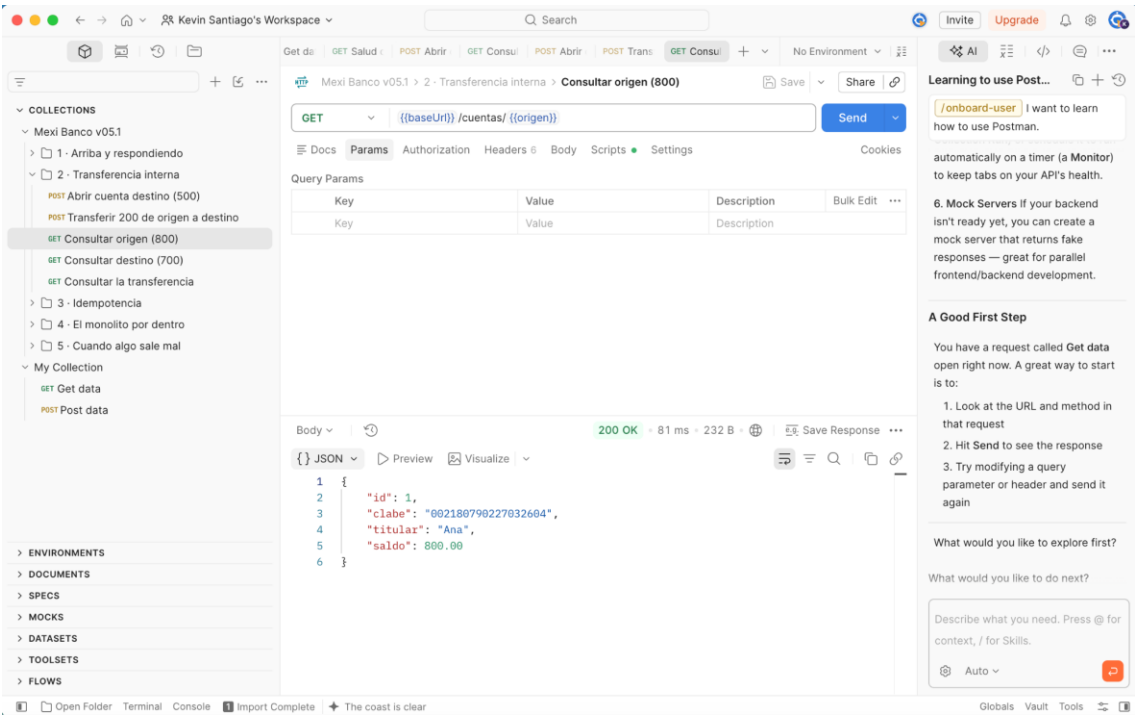


Figura 6. Saldo de origen después de transferir.

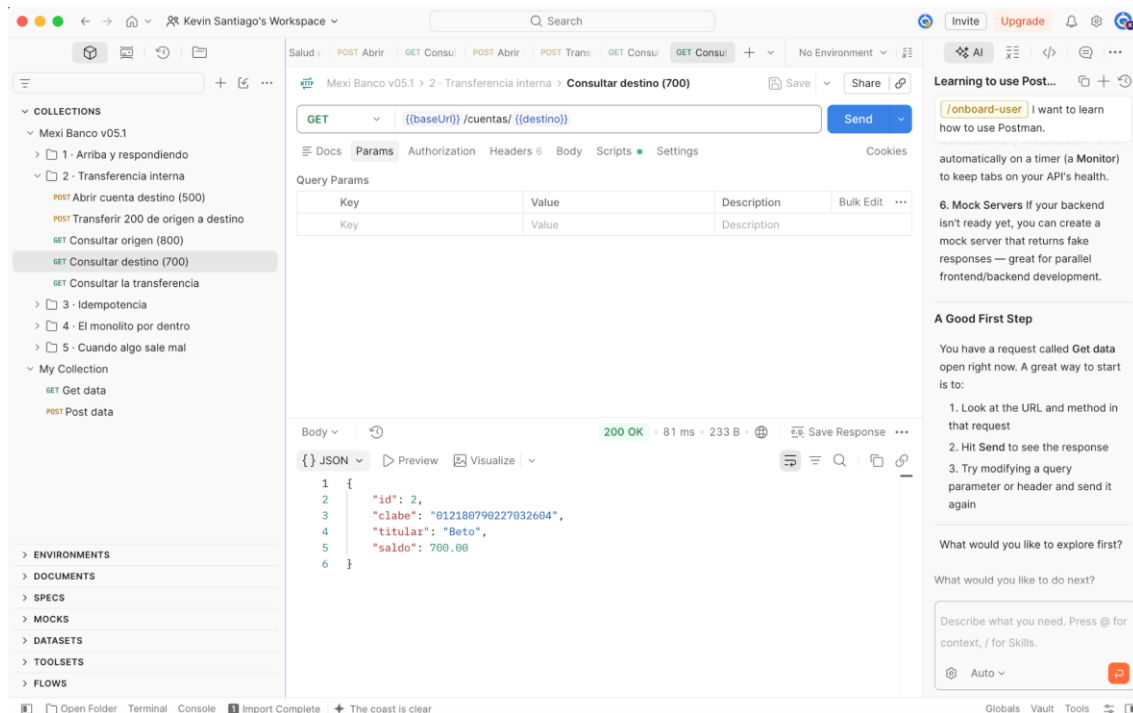


Figura 7. Saldo de destino después de transferir.

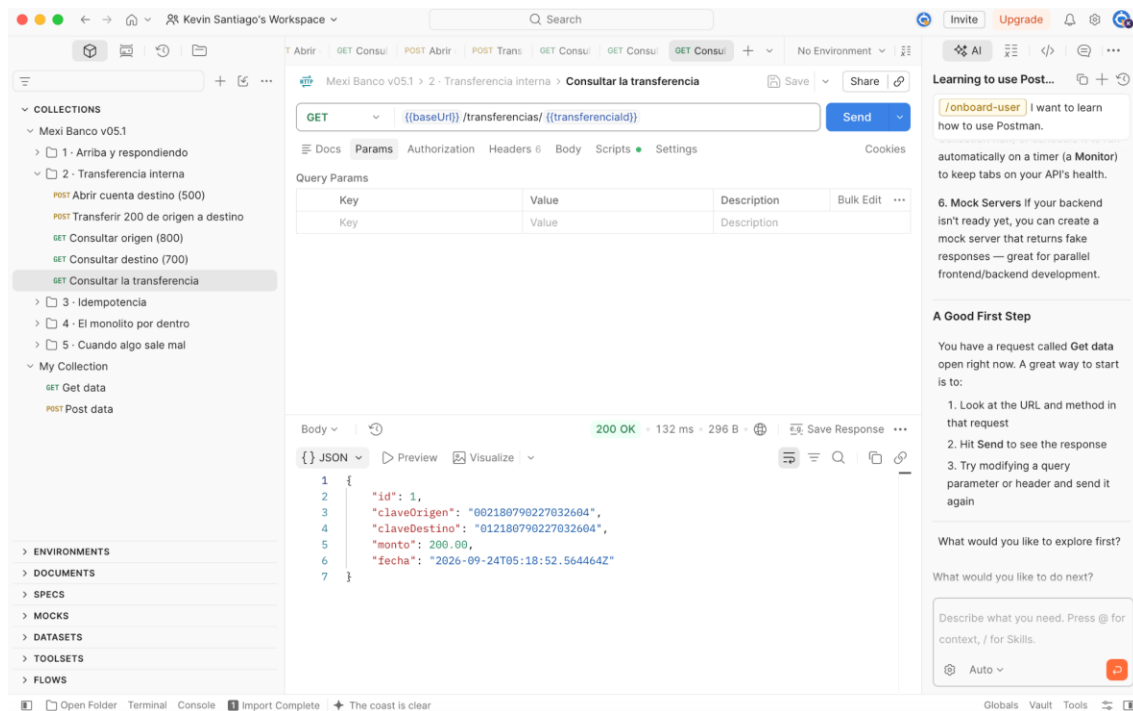


Figura 8. Consulta de la transferencia.

GET consulta y POST solicita una creación u operación. El cuerpo JSON contiene los datos. El código HTTP comunica el resultado y 201 confirma una creación. Para comprobar el efecto de la operación también hay que consultar el saldo y los registros correspondientes.

- **SPEI e idempotencia**

Se envió POST /spei con un monto de 50 y el encabezado Idempotency-Key. El primer envío respondió 201 Created y estado ENVIADO. El segundo envío utilizó la misma clave y devolvió el mismo identificador.

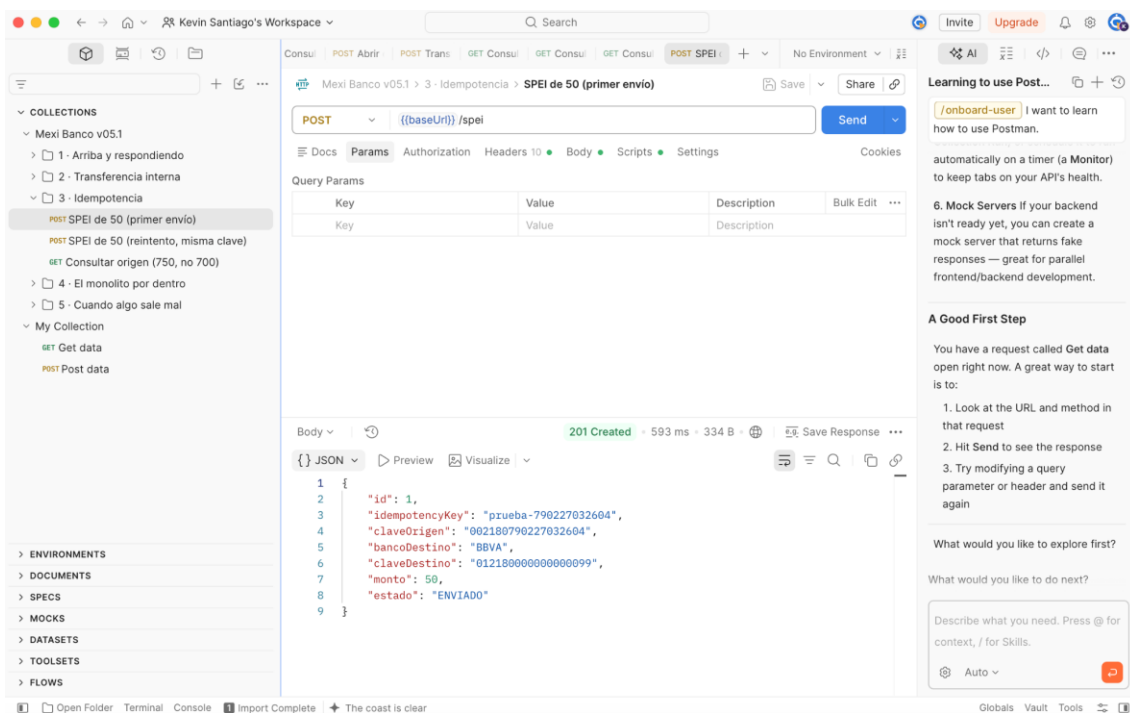


Figura 9. Primer envío SPEI. El envío de 50 devuelve 201 Created, un identificador y el estado ENVIADO.

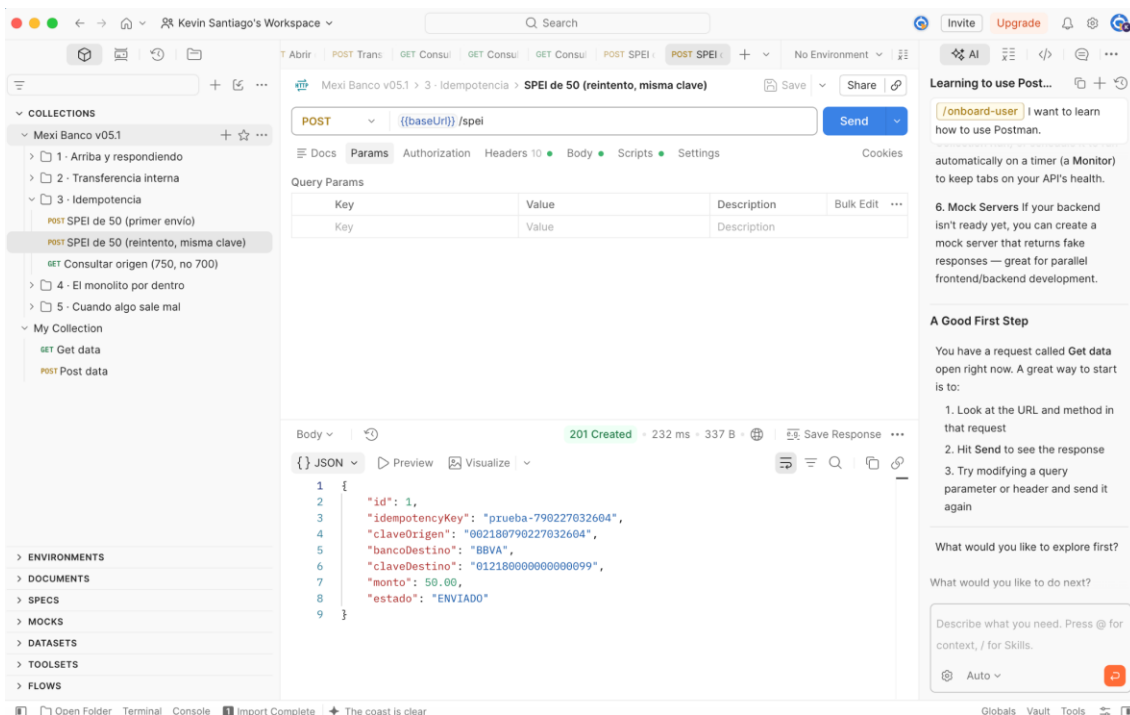


Figura 10. Reintento SPEI con la misma clave. La respuesta conserva el identificador del primer envío.

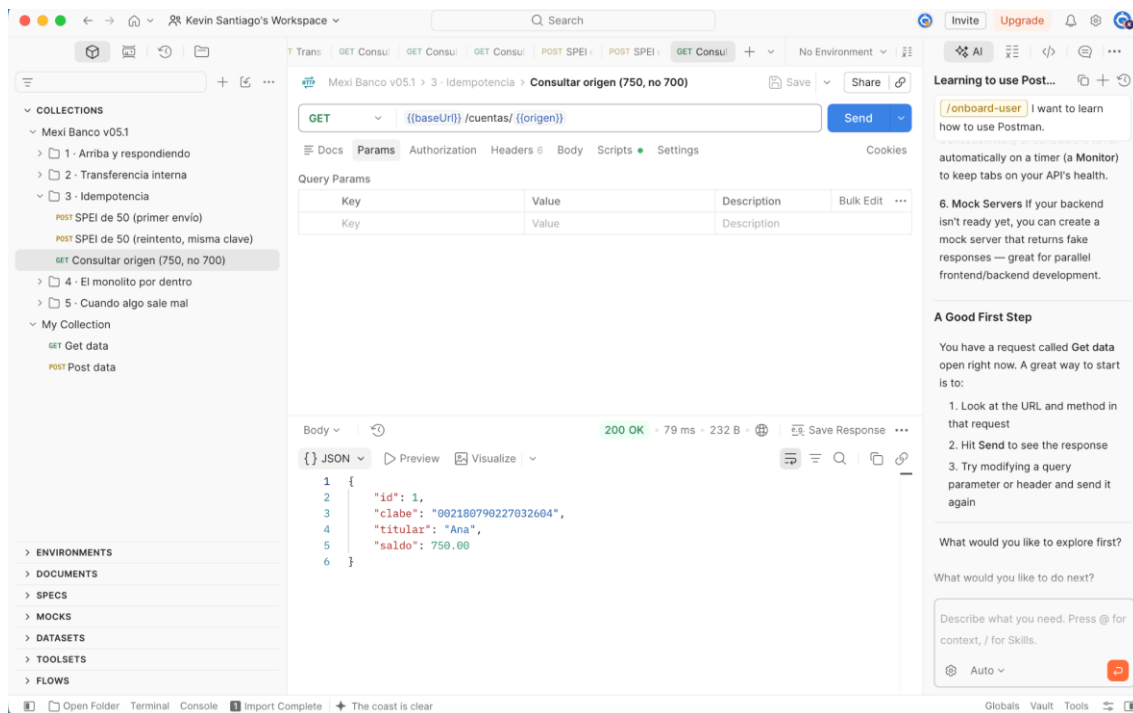


Figura 11. Saldo después del reintento.

La idempotencia evita que un reintento de la misma operación produzca un segundo cargo. El cliente puede repetir una solicitud porque no recibió la respuesta o perdió conexión.

• Movimientos y notificaciones

GET /movimientos?clabe={origen} devolvió los movimientos de la cuenta. Se observan un depósito inicial de 1000, un cargo de 200 por transferencia y un cargo de 50 por SPEI. Su suma es 750 y existe un solo SPEI_ENVIADO. GET /notificaciones?clabe={origen} mostró dos avisos: uno por la transferencia y otro por el SPEI.

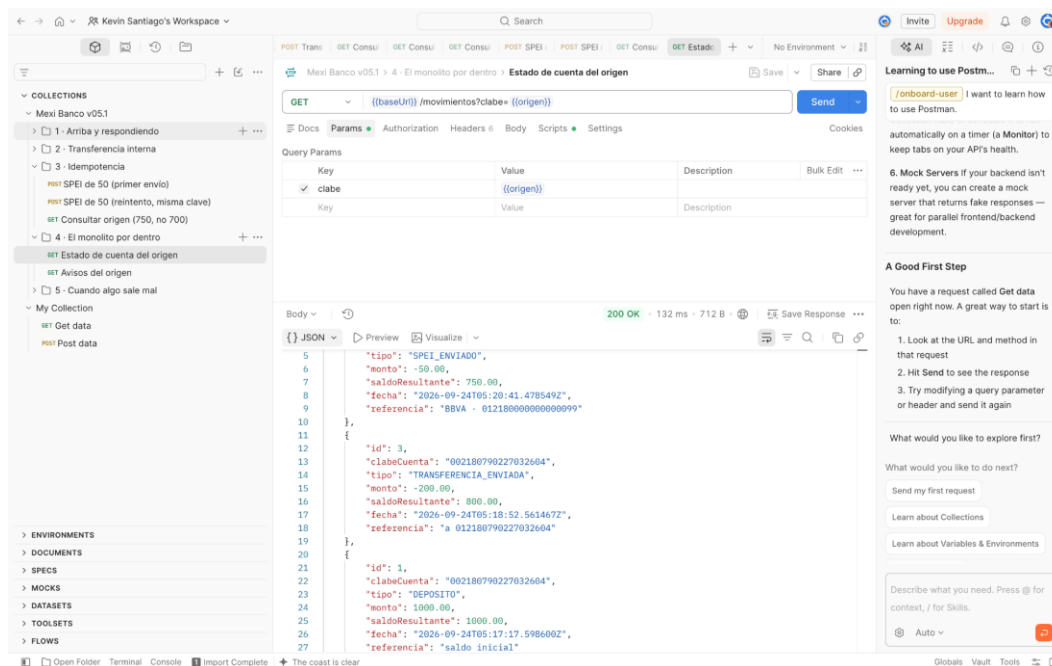
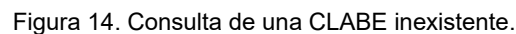


Figura 12. Estado de cuenta del origen. Se muestran el depósito inicial y los cargos por transferencia y SPEI.



1. Clabe inexistente – 404: No se encontró la cuenta solicitada.



2. Clabe repetida – 409: La apertura entra en conflicto con una cuenta existente.

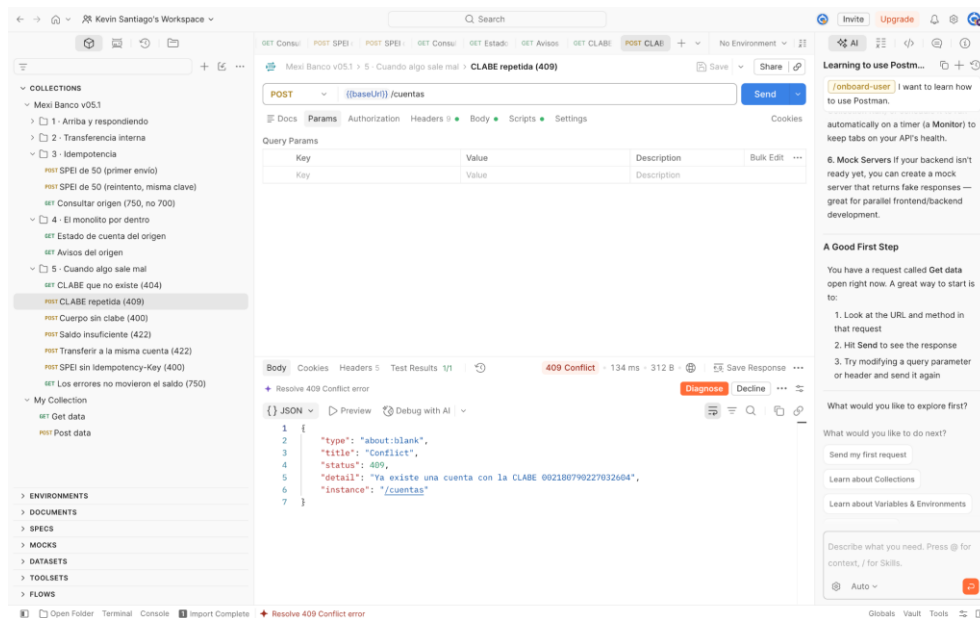


Figura 15. Intento de abrir una CLABE repetida.

3. Cuerpo sin cable – 400: La solicitud no cumple los datos obligatorios.

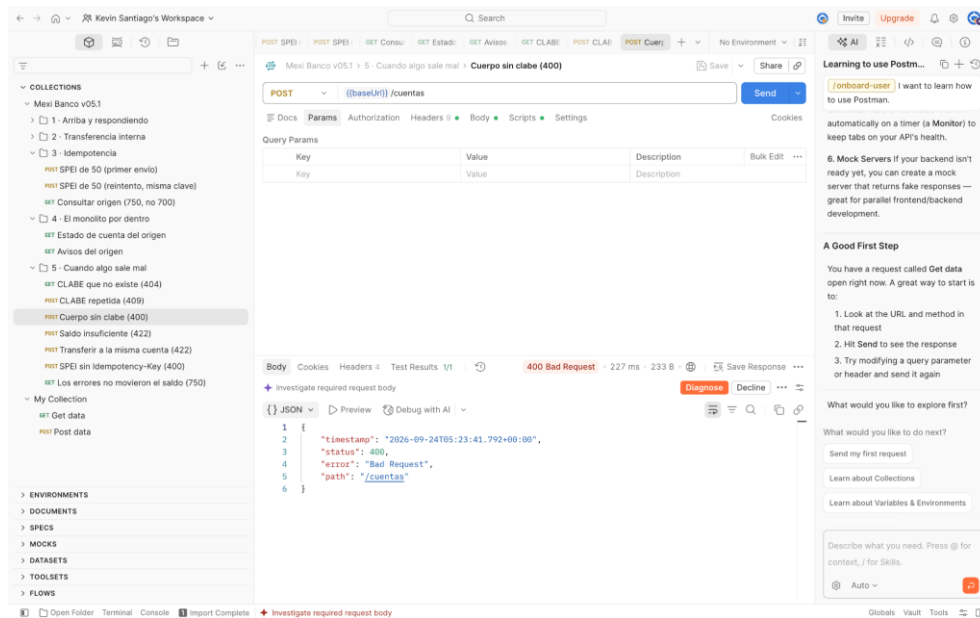


Figura 16. Solicitud sin CLABE.

4. Transferir a la misma cuenta – 422: Origen y destino no pueden ser la misma.

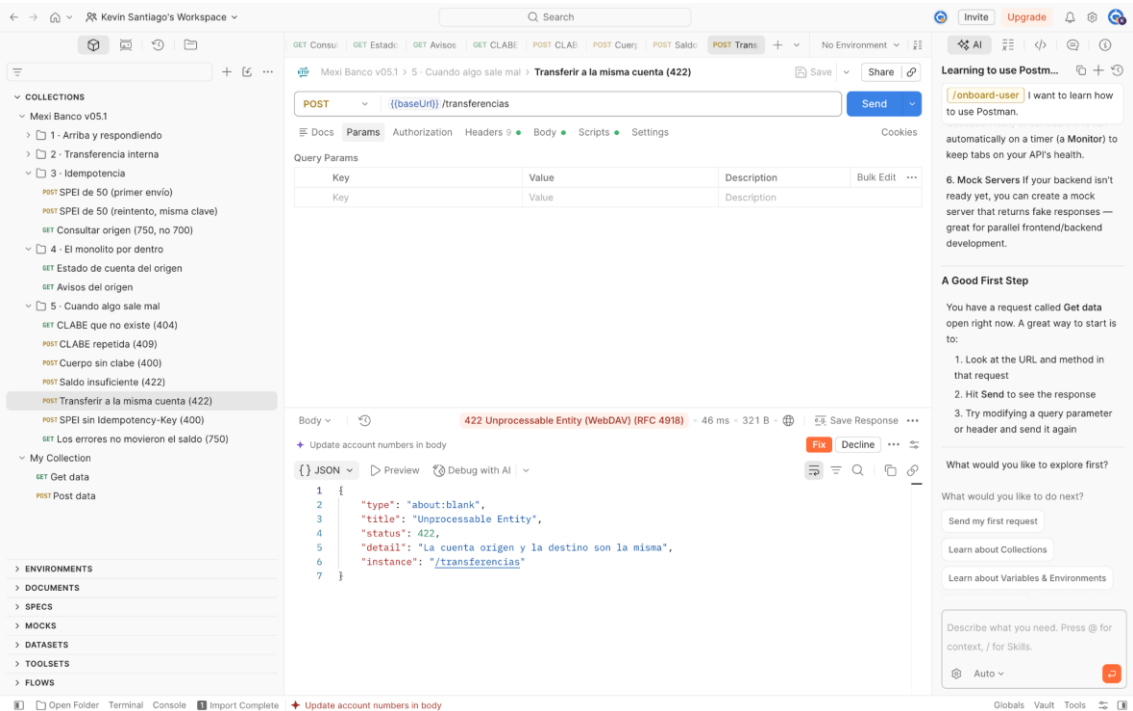


Figura 17. Transferencia a la misma cuenta.

5. Saldo insuficiente- 422

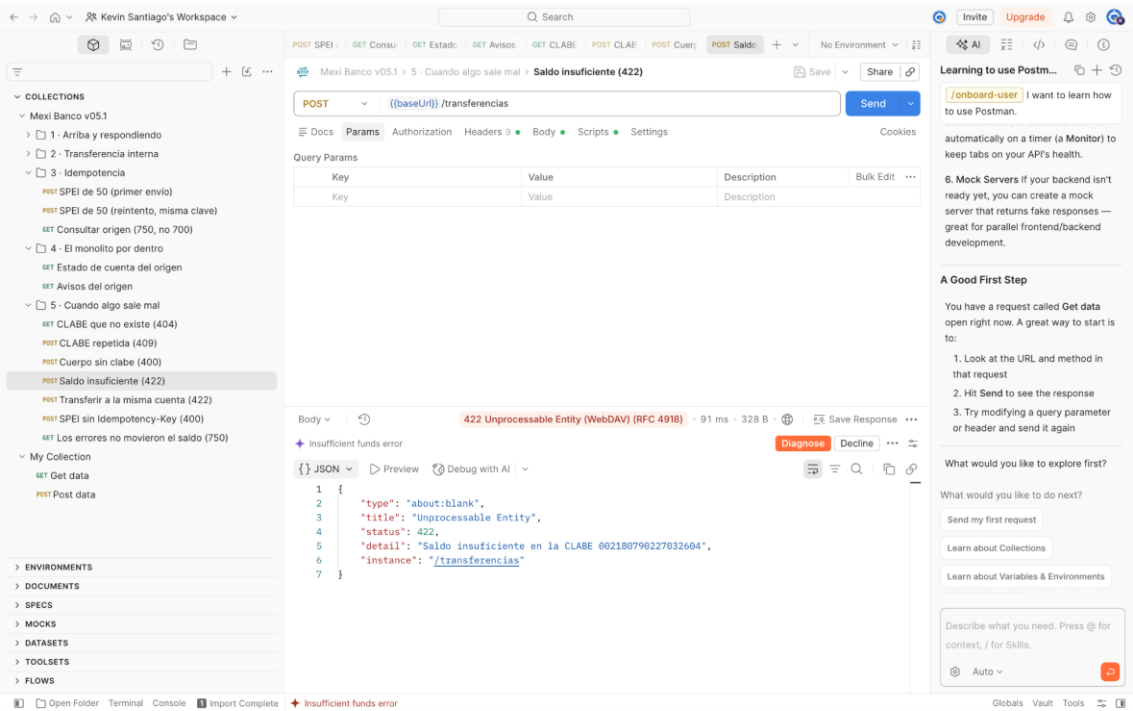


Figura 18. Transferencia con saldo insuficiente.

6. SPEI sin Idempotency-key – 400: Falta un encabezado.

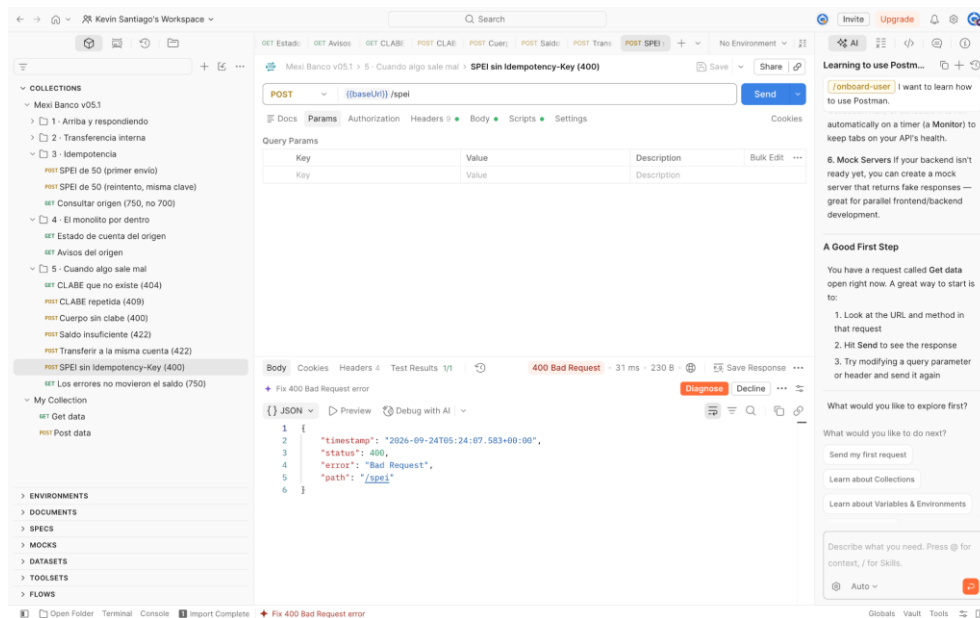


Figura 19. SPEI sin clave de idempotencia.

4. Persistencia con y sin volumen

Apagado conservando el volumen

docker

compose

down

docker compose up -d

down detiene y elimina los contenedores y la red pero conserva el volumen. Al iniciar otra vez, PostgreSQL utiliza los datos existentes.

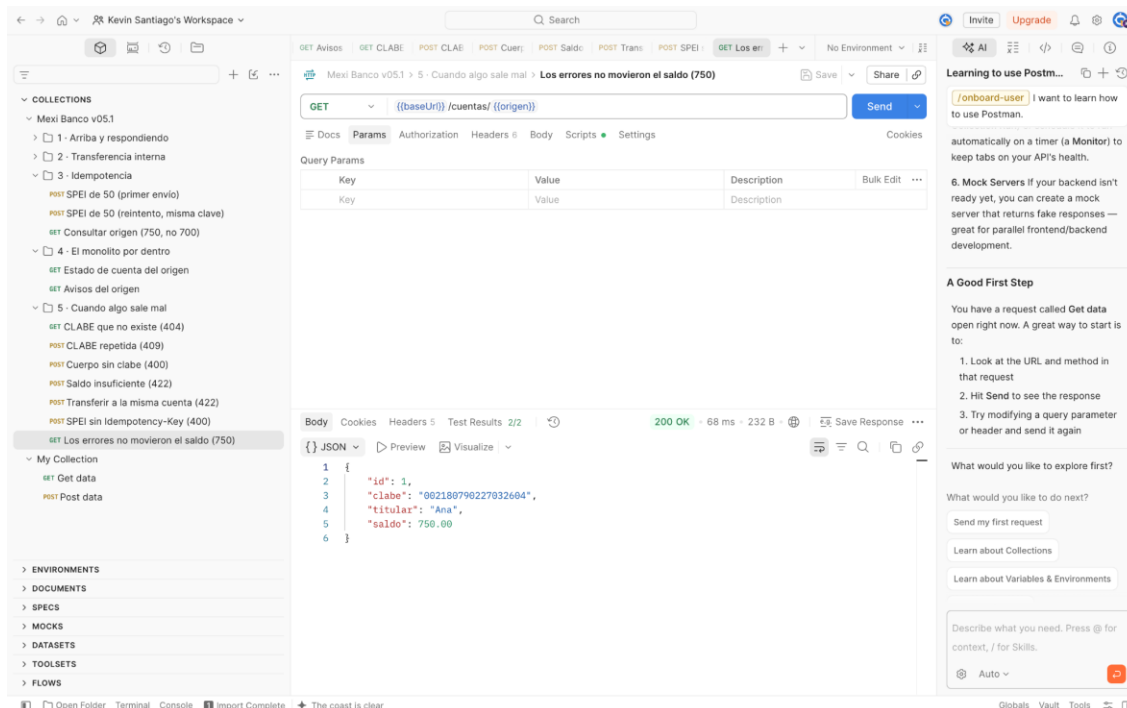


Figura 20. Consulta de persistencia con saldo conservado.

Apagado eliminando el volumen

docker compose down -v

docker compose up -d

La opción -v elimina el volumen nombrado de este proyecto. Al volver a iniciar, PostgreSQL debe inicializar una base vacía.

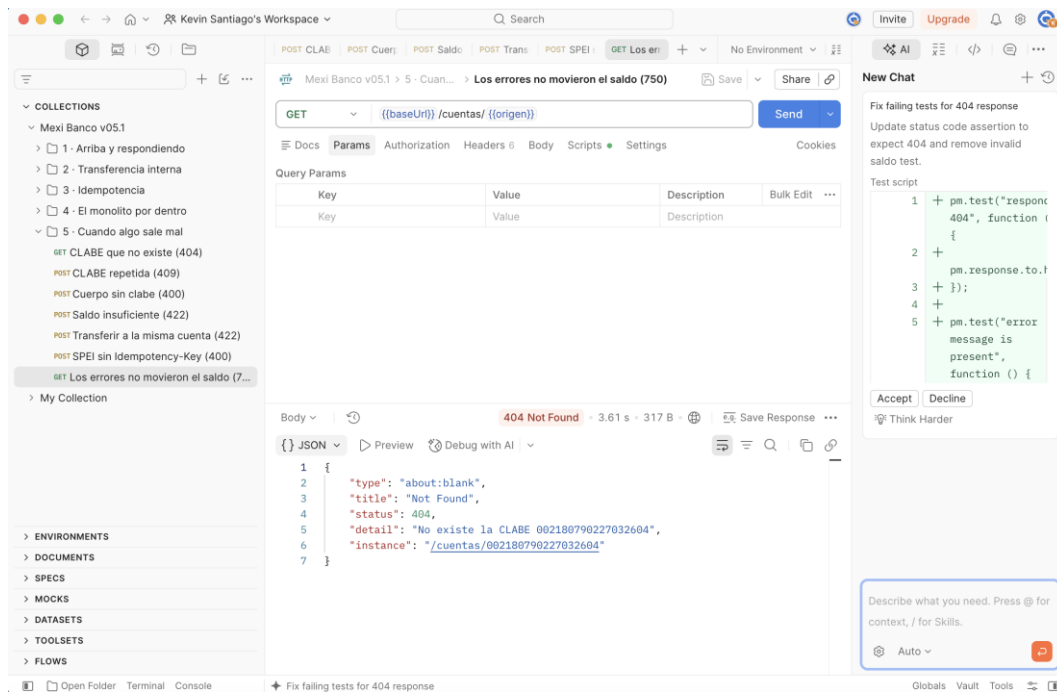


Figura 21. Consulta

5. Organización del monolito y sus capas

El código se organiza en `src/main/java/mx/mexibanco`. Los módulos cuenta, movimiento, transferencia, notificacion y spei pertenecen a una misma aplicación Java.

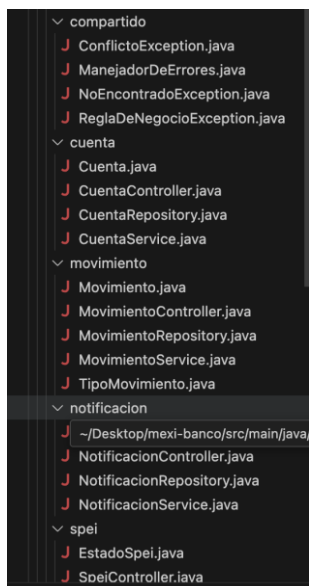


Figura 22. Estructura de cuentas y componentes compartidos.



Figura 23. Estructura de los otros módulos.

- **Apertura**

CuentaController usa `@RestController` y `@RequestMapping("/cuentas")`. Su método `abrir` recibe `POST /cuentas`, transforma el cuerpo JSON en `AltaCuenta` y llama a `cuentas.abrir`. `@ResponseStatus(HttpStatus.CREATED)` establece el código 201 para una apertura exitosa.

`AltaCuenta` es un record con `clabe`, `titular` y `saldoinicial`. `@NotBlank` exige texto no vacío para `CLABE` y `titular`. `@NotNull` pide un saldo.

`CuentaService.abrir` revisa `existsByClabe`. Si la `CLABE` existe, lanza `ConflictoException`. En caso contrario guarda una `Cuenta` nueva. Cuando el saldo inicial es positivo, llama a `movimientos.registrar` para registrar un `DEPOSITO` con ese monto y el saldo resultante.

- **Cargo y abono**

Cargar consulta la cuenta y compara el saldo con el monto. Si el saldo es menor, lanza `ReglaDeNegocioException`. Si alcanza, invoca `cuenta.aplicar(monto.negate())`, convirtiendo el monto en negativo para descontarlo. abonar consulta la cuenta y llama a `cuenta.aplicar(monto)`, aumentando su saldo.

- **Transactional**

`abrir`, `cargar` y `abonar` usan `@Transactional`. Esto agrupa operaciones sobre la base de datos. En una apertura, la cuenta y el movimiento inicial deben guardarse de manera coherente.

- **Módulo de cuentas**

`transferencia` y `SPEI` deben modificar saldos mediante `cargar` y `abonar`, sin acceder directamente a `CuentaRepository`.

6. Imagen con Dockerfile

```
Dockerfile > ...
1  # ---- ETAPA 1: CONSTRUCCION ----
2  FROM eclipse-temurin:21-jdk-alpine AS builder
3  WORKDIR /app
4  COPY pom.xml .
5  COPY mvnw .
6  COPY .mvn .mvn
7  RUN ./mvnw dependency:go-offline -B
8  COPY src ./src
9  RUN ./mvnw clean package -DskipTests
10
11 # ---- ETAPA 2: EJECUCION ----
12 FROM eclipse-temurin:21-jre-alpine
13 WORKDIR /app
14 COPY --from=builder /app/target/*.jar app.jar
15 EXPOSE 8080
16
17 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figura 24. Dockerfile

Etapas 1:

- **From ...:** Selecciona Java 21 con herramientas de desarrollo.
- **WORKDIR /app:** Establece el directorio de trabajo en la imagen.
- **COPY pom.xml . :** Copia la configuración del proyecto.
- **COPY mvnw . :** Copia el ejecutable de Maven Wrapper.
- **RUN ./mvnw dependency:go-offline -B:** Prepara dependencias con Maven en modo no interactivo.
- **COPY src ./src:** Copia el código fuente.
- **RUN ./mvnw clean package -DskipTests:** Limpia y empaqueta el proyecto en un JAR.

Etapas 2:

- **RUN ./mvnw clean package -DskipTests:** Limpia y empaqueta el proyecto en un JAR
- **FROM ...21-jre-alpine:** Inicia la etapa final con el entorno para ejecutar Java.
- **WORKDIR /app:** Define el directorio de ejecución de esta segunda etapa.
- **COPY --from=builder /app/target/*.jar app.jar:** Copia el JAR construido y le asigna el nombre app.jar.
- **EXPOSE 8080:** Documenta el puerto de la aplicación.

- ENTRYPOINT ["java", "-jar", "app.jar"]: Define el proceso que se ejecuta al iniciar el contenedor.

Docker Compose

Compose describe cómo ejecutar app y db, conectarlas, configurar la aplicación y conservar los datos.

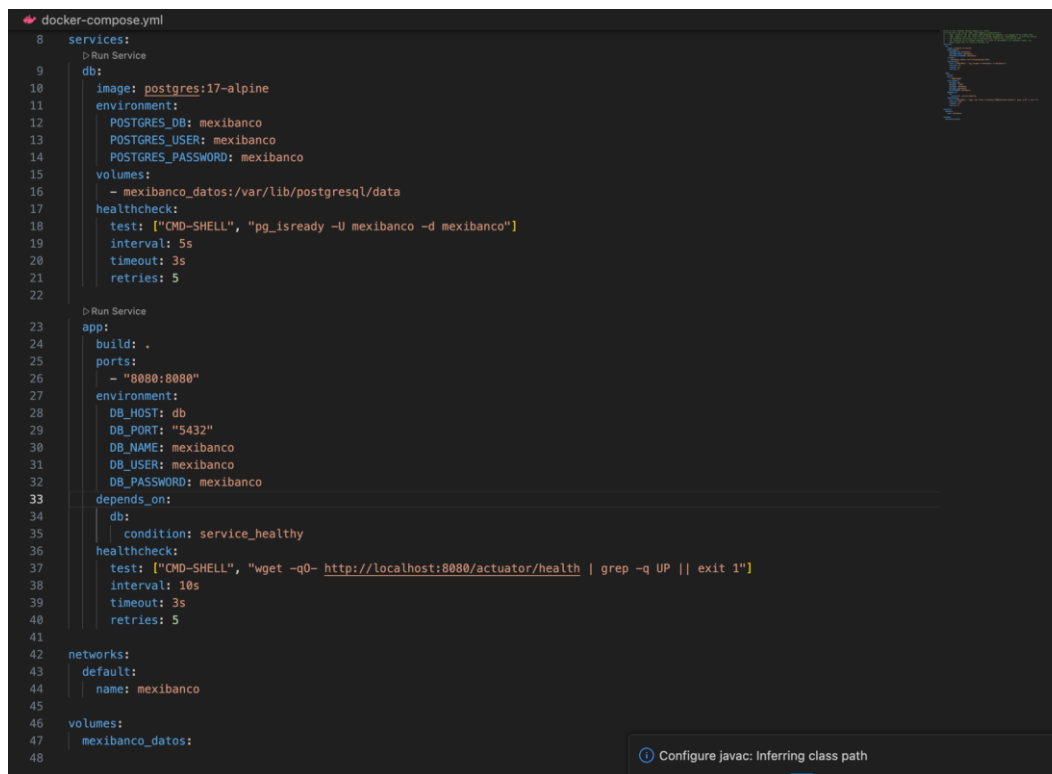


Figura 25. Configuración de Docker Compose.

- db.image: postgres:17-alpine: Ejecuta PostgreSQL 17 sobre una imagen basada en Alpine.
- POSTGRES_DB, USER y PASSWORD: Definen la base y credenciales al inicializar un directorio de datos vacío.
- mexibanco_datos:/var/lib/postgresql/data: Monta el volumen
- app.build: . : Construye la imagen desde el Dockerfile del directorio actual.
- ports: "8080:8080": Publica el puerto 8080 del contenedor en el puerto 8080 del equipo.
- DB_HOST: db y DB_PORT: "5432": Dirigen la conexión de la aplicación al servicio db dentro de la red.
- DB_NAME, DB_USER y DB_PASSWORD: Proporcionan a la aplicación los datos para conectarse a PostgreSQL

- `depends_on` con `service_healthy`: Espera a que la base de datos supere su healthcheck antes de iniciar app
- `networks.default.name: mexibanco`: Asigna nombre a la red compartida por ambos servicios.
- `volumes: mexibanco_datos`: Declara el volumen persistente utilizado por db.